Installation Guide

This document explains how to set up SkillBridge for local development and for hosting the platform on a production server.

Prerequisites

- Node.js 18 or later
- Docker and Docker Compose
- Git
- Redis or another session store for production deployments

Quick install script

After extracting the project, you can bootstrap a Docker-based environment with the bundled installer instead of running each command manually:

```
chmod +x install.sh
./install.sh development
```

The script performs the prerequisite check, copies any missing `.env` files from their `*.example` templates, builds the Docker images, starts PostgreSQL/Redis, runs database migrations and seeds, then brings the full stack online. When it finishes you can visit `http://localhost:3000` for the frontend, `http://localhost:5002/api` for the API, and `http://localhost:5050` for pgAdmin. Update the generated `.env` files with real secrets when you're ready and rerun `docker compose up -d --build` to pick up the changes.

For production deployments run the installer with sudo/root so it can request TLS certificates and update nginx:

```
sudo ./install.sh production yourdomain.com
```

The production workflow provisions certificates via `scripts/deploy_server.sh`, applies migrations/seeds, and starts the full stack in detached mode. Remember to disable the installation API (`INSTALL_API_ENABLED=false`) once setup is complete.

1. Clone the repository

```
git clone <repo-url>
cd Skillbridge
```

2. Configure environment variables

Backend

Copy the example file and adjust values as needed:

```
cp backend/.env.example backend/.env
```

Edit `backend/.env` and provide your secrets. `FRONTEND_URL` should match the domain where the frontend will run (defaults to `http://localhost:3000`). Leave `NODE_ENV` unset so cookies work over HTTP. If you need cross-subdomain cookies without HTTPS, also set:

```
COOKIE_SECURE=false
```

```
COOKIE_SAMESITE=None
```

A Redis instance (or compatible store) is required to persist sessions in production. Set `REDIS_URL` in `backend/.env` to point to your Redis server (for example `redis://localhost:6379`).

The book filter's price range uses configurable defaults:

```
BOOK_PRICE_RANGE_DEFAULT=100
BOOK_PRICE_RANGE_MAX=500
```

These values are read in `backend/src/config/books.js` and mirrored on the frontend via `frontend/src/utils/constants.js`. When running the frontend outside Docker, add the `NEXT_PUBLIC_` variants to `frontend/.env.local`:

```
NEXT_PUBLIC_BOOK_PRICE_RANGE_DEFAULT=100
NEXT_PUBLIC_BOOK_PRICE_RANGE_MAX=500
```

Initial admin passwords

The seed process provisions two built-in accounts: `SuperAdmin` and `Admin`. Set `APP_DOMAIN` in `backend/.env` first so the generated email addresses (`support@<APP_DOMAIN>` and `admin@<APP_DOMAIN>`) match your domain.

- `ADMIN_INITIAL_PASSWORD` (optional) lets you control the Admin user's password. If the variable is omitted a secure random password is generated and printed in the seed output.
- The SuperAdmin user is currently created with the default password `Javaheat@18880`. Change this password immediately after the first login, or edit `backend/src/seeds/seed_superadmin_user.js` before seeding if you prefer a different default.

Frontend (optional)

When using Docker Compose the frontend automatically points to the API on port `5002`. If you start the Next.js app separately, create `frontend/.env.local` and set:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:5002/api
NEXT_PUBLIC_TRUSTED_ICON_HOSTS=yourdomain.com,cdn.yourdomain.com
```

`NEXT_PUBLIC_TRUSTED_ICON_HOSTS` defines a comma-separated list of allowed hosts for payment method icons. URLs outside this list will fall back to a default icon.

The root `.env` file defaults `NEXT_PUBLIC_API_BASE_URL` to `http://backend:5002/api` so Docker services can reach the backend container internally during development. For production builds use `frontend/.env.production` instead and remove or override the root `.env` so the frontend points to your public domain (for example, `https://yourdomain.com/api`).

3. Install dependencies (optional)

For manual development outside of Docker:

```
cd backend && npm install
cd ../frontend && npm install
cd ..
```

4. Prepare the database

Run migrations and seed data:

```
cd backend
npm run migrate
npm run seed
cd ..
```

5. Launch the stack

Start all services with Docker Compose:

```
docker compose up --build          # Docker 20.10+
# or, if you still use the legacy plugin:
docker-compose up --build
```

The containers expose the following URLs:

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:5002/api`
- PostgreSQL: `localhost:5432`
- pgAdmin: `http://localhost:5050`
- Redis: `localhost:6379`

Once running, open the frontend URL in your browser and create an account or log in.

Validate your installation

> ***Checklist:*** *Run these quick checks to confirm the Docker stack and database are ready for day-to-day use.*

- Visit `http://localhost:3000` (or your domain) to confirm the Next.js frontend loads without console errors.
- Sign in with the admin account created during `install.sh`, then open `/dashboard/admin` to verify you can reach the admin panel.
- Hit the API health endpoint at `http://localhost:5002/api/health` (or `https://<domain>/api/health`) and ensure it reports `status: ok`.
- Check the backend logs with `docker compose logs backend` for migration errors or SMTP failures before sharing the environment.

6. Running tests

The project includes Jest suites for both the API and the frontend.

Backend

```
cd backend
npm test
```

Frontend

```
cd frontend
npm test
```

7. Hosting on a server

To deploy SkillBridge for real users on a remote host:

1. **Provision a server** running Linux and install Docker and Docker Compose.
2. **Clone the repository** on the server and configure environment variables as described above.
   - In `backend/.env`, set production values such as `NODE_ENV=production`, `FRONTEND_URL=https://<your-domain>`, `COOKIE_SECURE=true`, and `COOKIE_SAMESITE=None`.
   - Create `frontend/.env.local` with `NEXT_PUBLIC_API_BASE_URL=https://<your-domain>/api`.
3. **Adjust Nginx for your domain.**
   - Set the `APP_DOMAIN` environment variable so Nginx and the backend use your domain.
   - Obtain TLS certificates (e.g. via Let's Encrypt's certbot) and ensure the paths in `ssl.conf` match the certificate locations.
4. **Run database migrations and seeds** before starting the containers:

   ```
   docker compose run --rm backend npm run migrate
   docker compose run --rm backend npm run seed
   # Legacy plugin:
   # docker-compose run --rm backend npm run migrate
   # docker-compose run --rm backend npm run seed
   ```

5. **Build and start the containers** in detached mode:

   ```
   docker compose up -d --build
   # Legacy plugin: docker-compose up -d --build
   ```

6. **Verify the deployment** by visiting `https://<your-domain>` in a browser. The API will be available at `https://<your-domain>/api`.

For updates, pull the latest changes and rebuild:

```
git pull
docker compose up -d --build
# Legacy plugin: docker-compose up -d --build
```